

ColdTrack AI — Evaluation Artifact (Baseline)

AIC COMPFEST 18 Final · Smart Logistics · version tag: **baseline**

What is this document?

ColdTrack AI watches the sensors of a refrigerated truck and warns the operator before the cargo gets too warm. This document reports how well the **prototype from the qualifying round** does that **today, before any improvement**. We tested it the way an operator would use it: feed in the last 60 minutes of sensor readings, then look at the status (**AMAN** = safe, **WASPADA** = warning, **KRITIS** = critical) and at the countdown to the moment the cargo leaves its safe temperature range. The test used 106 simulated trips (16,244 one-hour snapshots) that the model never saw while it was being trained.

Key terms

Term	Meaning
Breach	The cargo temperature goes outside its safe range (for example above 8 °C for vaccines).
Countdown (time-to-breach)	Minutes left before a breach. This is the main feature of the product.
Window	One snapshot: the last 60 minutes of sensor readings from one truck.
Imminent breach	A window where a breach really happens within 30 minutes (this is when the operator must act).
Fault types	A0 = healthy · A1 = door left open · A3 = cooling unit fully off · A7 = sudden extreme outside heat · A8 = poor pre-cooling · degradasi = cooling slowly getting weaker · masalah_sensor = faulty temperature sensor.

Summary

The prototype passed **8 of 23** checks. It is good at predicting temperature, and the scores we reported in the qualifying round are confirmed (fault-identification score 0.58, forecast error 0.20 °C). However, when we looked at what an operator would actually see, the system often **fails to raise the alarm when it matters most**:

#	What we found	Why it matters	Severity
F1	When the cargo was less than 30 minutes from a breach, the system still showed green (AMAN) in 32% of cases (about 1 in 3). It showed warning in 43% and critical in only 25%.	A false “all clear” is the most dangerous mistake for a cold chain.	Critical
F2	The countdown is often missing when it is needed. The two AI models alone show it in only 25% of urgent cases (62% once the backup rule in the server is added). Reason: the system first decides if the truck is healthy, and it wrongly said “healthy” for 68% of the urgent cases, which hides the countdown.	The main feature of the product does not appear in many urgent situations.	Critical
F3	Sensor noise confuses it. With small random noise (0.3 °C, common in real sensors) urgent cases judged “healthy” rise from 68% to 82%. Our training data had almost no noise (0.05 °C).	Real trucks have noisier sensors than our simulator.	High
F4	A frozen (stuck) temperature sensor is read as “healthy”: 73% of such cases, and only 10% are flagged as a sensor problem.	A stuck sensor hides real warming.	High
F5	The temperature-forecast score looks good but proves little. Simply guessing “the temperature stays the same” has an error of 0.195 °C, which is as good as our AI (0.198 °C). In almost all test snapshots the temperature barely moves (only 84 of 16,244 change by 1 °C or more).	The forecast target we passed does not show the AI is useful.	High
F6	Three fault types are mostly missed: sudden outside heat (A7) is found 44% of the time, poor pre-cooling (A8) 20%, and slowly weakening cooling (degradasi) only 11%.	These faults would go unnoticed.	High
F7	Our test data is too small to prove much. Some fault types appear in only 2–4 test trips, and there are no urgent cases for vaccine, frozen meat or fish.	Some results could be luck, and the vaccine use case is untested.	Medium

1. How we tested

Idea: test what the **user really sees** (status, countdown, fault type), not only the AI models inside. We built 5 groups of tests and 23 pass/fail checks. Each check states a minimum we consider acceptable for a real fleet operator.

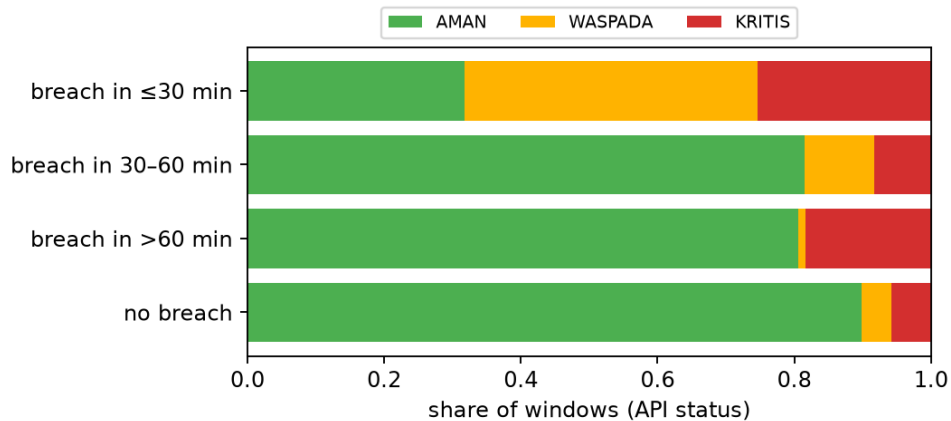
Test group	What we did	What we measured	What this can NOT tell us
1. Temperature forecast	Compared the predicted temperature 15, 30 and 60 minutes ahead with what really happened in 16,244 test snapshots. Also compared with a “no change” guess.	Average error in °C, worst-case error, and error only on snapshots where the temperature really changes.	Most snapshots are calm, so the average hides performance when it matters. It also says nothing about whether an alarm would ring.
2. Fault identification	Asked which of the 7 conditions each snapshot shows. Repeated the scoring 200 times on random samples of trips to see how uncertain the numbers are.	How many real faults of each type were found, how many alarms were correct, and a confusion chart.	A fault type with only 2–3 test trips cannot be judged reliably.
3. Urgent-breach warnings	Took only snapshots where the cargo really was 30 minutes or less from a breach, and checked whether the countdown was shown and how accurate it was.	Share of urgent cases with a countdown, average countdown error, and why the countdown was missing.	In qualifying we reported “7 min error” for these cases, but that ignored the cases where no countdown appeared at all.
4. Stress tests	Added sensor noise (0.1 / 0.3 / 0.5 °C), shifted the clock by 7 hours (a time-zone mix-up), and froze the temperature sensor during a real warming.	Fault-identification score, and how many urgent cases are judged healthy.	The noise is artificial, so this shows sensitivity, not the exact error on real trucks.
5. Whole-system check	Sent 8,122 snapshots through the real server (the same request the app sends) and read the status and countdown like a user.	Status by how close the breach really is, countdown shown, response time, and match with the raw AI models.	The “correct answer” comes from our simulator, not from real trucks.

Existing checks from the qualifying round (run for reference, not graded here): the server tests pass 24/24 and the data-format tests pass 7/7. Simple comparison models (XGBoost, linear, Isolation Forest) re-run on the same test data score about the same as our AI on the forecast (error 0.19 °C) and better on fault identification (0.67 vs 0.58).

Limits of this evaluation: only 106 test trips (27 with a fault); all data is simulated; the pass marks are our own choices, set after seeing the qualifying numbers, so they are open to challenge. Technical details of every test are in Appendix A.

2. What we found (where the system fails, with evidence)

F1 · Urgent cases shown as “safe”



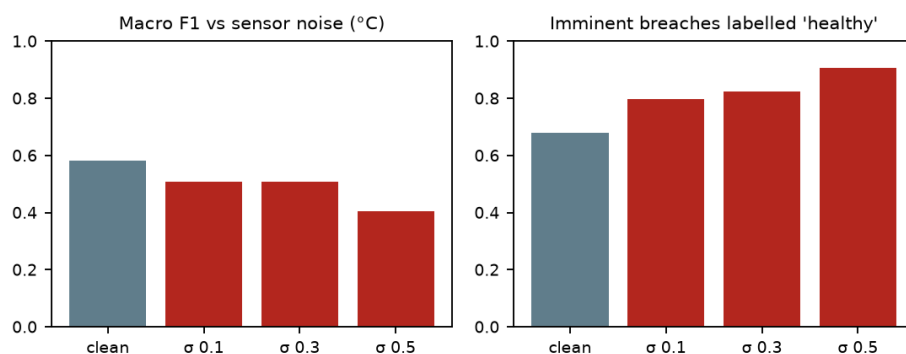
How to read the chart: each bar is a group of snapshots, grouped by how soon the cargo *really* leaves its safe range. Colors show what the system displayed. The top bar (breach within 30 minutes) should be almost all red, but a third of it is green.

Why: the status mostly follows the temperature forecast, which is roughly “no change” (F5). A rule that raises the status when a countdown exists does not help, because the countdown is hidden whenever the system believes the truck is healthy. When the countdown is shown it is accurate (average error 1.6 minutes), so the problem is **missing warnings, not wrong numbers**. Also, only 19% of snapshots 30–60 minutes before a breach get any warning, so the 60-minute early-warning setting never really triggers.

F2 · The countdown depends on the weakest part

Urgent cases (breach within 30 min, n=625)	Share
Countdown shown by the two AI models	25%
Hidden because the system judged the truck “healthy”	68%
Hidden because the system judged “faulty sensor”	5%
Hidden because the countdown was above the 30-minute display limit	3%
Average countdown error when shown / if a missing countdown counts as “30 min”	3.4 min / 18.3 min

F3 / F4 · Noise and a stuck sensor



Left: the fault-identification score drops as we add noise. Right: the more noise, the more urgent cases are wrongly judged “healthy”. When the temperature sensor freezes during a real warming, 73% of cases are judged healthy and only 10% are flagged as a sensor problem.

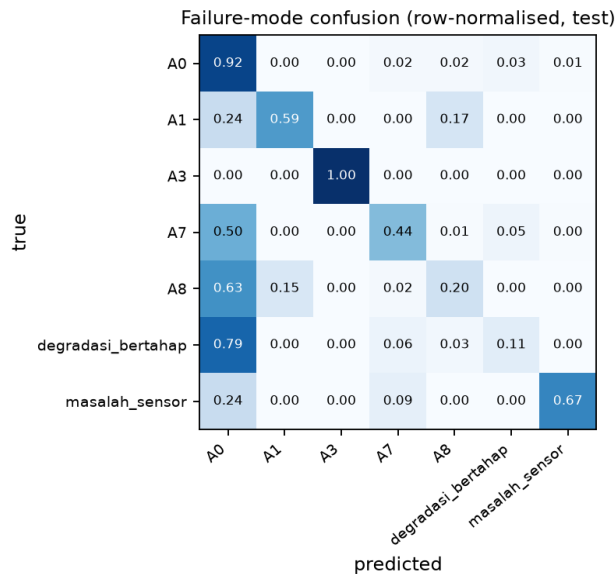
Two things we checked and found **fine**: (1) a 7-hour clock shift (time zone) barely changes results (score 0.59 vs 0.58); (2) the server gives the same answers as the raw AI models (average difference 0.005 °C), so nothing is lost between them.

F5 · The forecast score is not a strong proof

Average error of the 30-minute forecast (°C, lower is better)	All snapshots	Only snapshots where temperature changes (n=84)
Our AI (deployed)	0.198	1.24
Guess “same temperature as now”	0.195	1.56
Continue the trend of the last 5 minutes	0.614	2.65

Our target (error below 0.8 °C) was met, but the “no change” guess meets it too. The honest result is on the few snapshots where the temperature really moves: there our AI is about 20% better than the “no change” guess.

F6 · Which faults are found



How to read the chart: each row is the real condition, each column is what the system said. A perfect system has 1.00 on the diagonal. Rows for A7, A8 and degradasi are mostly spread into the “A0” (healthy) column, meaning these faults are called healthy.

Fault type	Test snapshots	Test trips	Alarms that were correct	Real faults found
A0 · healthy	12546	95	87%	92%
A1 · door open	321	4	64%	59%
A3 · cooling unit off	379	7	100%	100%
A7 · sudden outside heat	642	5	41%	44%
A8 · poor pre-cooling	651	6	30%	20%
degradasi · weakening cooling	833	5	16%	11%
masalah_sensor · faulty sensor	872	11	89%	67%

An odd pattern: after a fault starts, the system finds it best 30–120 minutes later (about 76%) and worst after 2 hours (38%). So “the fault is too new to see” does not explain the misses. We do not yet know the reason (see section 4).

F7 · The test data is too thin

Urgent cases (breach within 30 minutes) exist in the test data only for vegetables/fruit (319 snapshots) and dairy (306). There are none for vaccine, frozen meat or fish, so the main vaccine use case has no evidence for urgent cases. The uncertainty of the overall fault-identification score is large: somewhere between 0.49 and 0.64 (95% range).

3. Before and after improvement

Status: **baseline only**. The “After” column stays empty until an improvement is built and measured with the same tests.

Planned improvement 1 (in progress, not yet measured)

- **Use real inputs instead of made-up ones.** Real outside temperature and humidity for Indonesia (BMKG, 2010–2020); real product heat tables (fish, meat, vegetables, fruit) with the payload weight entered by the user; and real faulty-sensor recordings (Intel Berkeley Lab) to copy real sensor errors. This lets the product handle a wider range of goods.
- **Simulate the truck with physics.** Heat enters from the outside air, and a heavier load warms up more slowly.
- **Focus on 4 situations:** healthy, door open, sudden extreme outside heat, and faulty sensor. Cooling unit fully off (A3) and weakening cooling (degradasi) are postponed to a later version. Poor pre-cooling (A8) becomes a simple check before the truck leaves.

Because of this, the “After” results cover only 4 situations, and the fault-identification score over 4 situations cannot be compared directly with the 7-situation baseline.

What we measure (test data)	Before (baseline)	After	Comment
Checks passed	8 of 23	—	—
Urgent cases shown as “safe” (whole system)	32%	—	Goal: 5% or less
Urgent cases with a countdown (whole system)	62%	—	Goal: 80% or more
Urgent cases judged “healthy” by the AI	68%	—	Goal: 10% or less
Fault-identification score	0.58 (7 situations)	—	After: 4 situations, not directly comparable
Faults found: weakening cooling / poor pre-cooling / outside heat	11% / 20% / 44%	—	First two postponed or moved to a rule; only outside heat is compared
Fault-identification score with 0.3 °C sensor noise	0.51	—	Goal: drop of 0.05 or less
Frozen sensor flagged as sensor problem	10%	—	Goal: 50% or more
Forecast error where temperature really changes	1.24 °C	—	—

4. Findings not yet fixed — causes and plan

Causes below are **hypotheses** unless marked measured; each will be confirmed or rejected with the suite before any change.

Finding	Suspected cause (status)	Planned fix and how we will judge it
F1/F2 green status & silent TTB	TTB and status floors are gated on the classifier / forecast; both are weak on slow faults (measured: 68% of imminent windows → A0).	Adjust TTB guard to apply regardless of class. Judge: F1 AMAN-rate ≤ 5% without raising healthy WASPADA+KRITIS above 10%.
F3 noise fragility	Trained only on $\sigma=0.05$ °C simulator noise (dataset card, measured). Aggregate features (std, trend) are noise-sensitive (hypothesis).	Retrain on the physics-simulated trips driven by real BMKG outside-air data, with sensor errors from real Intel Berkeley Lab traces instead of $\sigma=0.05$ °C noise; re-run G4. Judge: F1 drop ≤ 0.05 at $\sigma=0.3$ (metrics).
F4 stuck sensor	Frozen temp looks like a stable healthy trace (hypothesis). masalah_sensor recall only 67% even in-distribution (measured).	Train masalah_sensor on real faulty-sensor traces (Intel Berkeley Lab), plus an explicit flat-line detector in preprocessing (less than X variance over last N minutes while ambient/reefer state changes) as a rule, independent of the network. May remain unresolved or removed, will be reported.
F5 forecast metric	Metric choice, not a model defect (measured).	Report the moving-window MAE and skill vs persistence as the primary forecast metrics.
F6 A7/A8/degradasi	Label taken at window end; slow faults look healthy early (partly measured: recall is low even 120+ min in, so not only onset latency). Class overlap A0↔A8/A7/degradasi in confusion matrix (measured).	Scope change: A8 (poor pre-cooling) leaves the classifier and becomes a pre-departure rule; degradasi_bertahap and A3 (total reefer failure) are deferred as a later feature. A7 (extreme ambient) stays: re-simulate it from real BMKG extremes, inspect misclassified windows, consider longer context. May remain unresolved, will be reported.

Finding	Suspected cause (status)	Planned fix and how we will judge it
F7 test validity	700 simulated trips; 106 in test (measured).	Regenerate a larger test set from the physics simulator, stratified by cargo (fish, meat, vegetables, fruit, from the product thermal tables) and payload mass; report CIs. Vaccine and dairy are not in the planned thermal tables, so claims for them stay unclear. Real inputs make the simulation more realistic but still cannot substitute for field data.

Appendix A — technical detail of the test groups

System under test: coldtrack.onnx (GRU) + coldtrack_ttb.onnx (XGBoost) + backend rule engine, evaluated as deployed on the held-out test split of dataset v4 (split by trip). Reproduce: `python -m eval.run_eval --tag baseline; pytest eval/test_baseline.py`.

Group	Test cases	Metric(s)	Why this metric / what it does NOT capture
G1 Forecast	sliced by fault mode and cargo; compared to naive persistence and linear extrapolation	MAE t+15/30/60, p95 and max error, MAE on windows that really move	MAE is dominated by the ~99% of windows where nothing changes; hence the persistence comparison and the moving-windows slice.
G2 Classification	7 classes; bootstrap over trips (200×) for CIs; recall vs minutes since fault onset	per-class P/R/F1, macro-F1, PR-AUC, confusion matrix	Macro-F1 hides which fault is missed; windows of one trip are highly correlated, so window-level CIs would be far too narrow.
G3 Imminent breach	windows with true TTB $\leq 10 / \leq 30$ min, gated exactly like backend/app/inference.py	TTB shown-rate, MAE when shown, miss reasons	The qualifying-round 'MAE 7.08 min for TTB ≤ 30 ' conditions on the true answer and ignores silent misses.
G4 Robustness	sensor noise $\sigma \in \{0.1, 0.3, 0.5\}$ °C; +7 h hour shift; stuck sensor injected into imminent-breach windows	macro-F1, forecast MAE, imminent-breach A0 rate	Training data is a clean simulator ($\sigma=0.05$ °C); perturbations are synthetic, so they show sensitivity, not real-world error.
G5 End-to-end API	every 2nd test window POSTed to /api/v1/analyze via FastAPI TestClient	status × true-TTB bucket, TTB shown, latency, parity with raw ONNX	Ground truth = simulator time_to_breach (threshold = cargo max limit; equals config.yaml).

Also: XGBoost/Linear/IsolationForest baselines (ml/baselines.py) re-run on the test split: forecast MAE@30 linear 0.192 / XGB 0.190; macro-F1 XGB 0.667; PR-AUC XGB 0.749, IsolationForest 0.352. The committed baseline_metrics.json is from the v3 validation split and is stale.

Appendix B — acceptance checks (baseline)

Result	Check
PASS	test_forecast_mae_t30_under_0_8C
PASS	test_recall_per_fault_class_at_least_0_5[A1]
PASS	test_recall_per_fault_class_at_least_0_5[A3]
PASS	test_recall_per_fault_class_at_least_0_5[masalah_sensor]
PASS	test_timezone_shift_does_not_change_macro_f1_much
PASS	test_healthy_windows_not_over_alerted
PASS	test_api_no_http_errors_and_latency_p95_under_300ms
PASS	test_api_matches_raw_onnx_forecast
FAIL	test_forecast_worst_case_error_bounded
FAIL	test_macro_f1_at_least_0_80
FAIL	test_recall_per_fault_class_at_least_0_5[A7]
FAIL	test_recall_per_fault_class_at_least_0_5[A8]
FAIL	test_recall_per_fault_class_at_least_0_5[degradasi_bertahap]
FAIL	test_anomaly_recall_at_least_0_8
FAIL	test_imminent_breach_ttb_shown_directly_by_models_at_least_80pct
FAIL	test_imminent_breach_not_gated_as_healthy
FAIL	test_test_split_covers_every_cargo_profile_for_imminent_breach
FAIL	test_sensor_noise_0_3C_macro_f1_drop_at_most_0_05
FAIL	test_sensor_noise_0_3C_does_not_hide_imminent_breach
FAIL	test_stuck_sensor_during_breach_is_flagged_not_healthy
FAIL	test_imminent_breach_never_green_more_than_5pct
FAIL	test_imminent_breach_ttb_shown_by_api_at_least_80pct
FAIL	test_every_fault_mode_has_at_least_5_test_trips

Commit of the code that produced these numbers: f351c03 (+ uncommitted eval/ at generation time). Dataset: data/processed/v4_seed1000_700trips.parquet, split=test. Seeds fixed at 42.